

ESCUELA:

Tecnológico Nacional de México campus Culiacán

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Tópicos de Inteligencia Artificial

Profesor(a):

Zuriel Dathan Mora Felix

Nombre de la tarea:

Tarea Bonus Módulo II

ALUMNOS:

22170574 Bejarano Ureta Sebastián

22170861 Garcia Inzunza Kevin Alejandro

FECHA:

09/10/2026

Culiacán Sinaloa

INTRODUCCIÓN

En este documento se aborda el estudio de cuatro problemas representativos dentro de la teoría de algoritmos y la optimización combinatoria: el Problema de Programación de Trabajos (JSSP), el Problema de las N Reynas, el Árbol de Expansión Mínima (MST) y el Problema del Agente Viajero (TSP). La resolución de estos planteamientos es fundamental en las ciencias de la computación para modelar y resolver situaciones complejas del mundo real, tales como la asignación de recursos compartidos a lo largo del tiempo, diseño eficiente de redes de telecomunicaciones e infraestructuras y la optimización de rutas logísticas para minimizar costos de traslado. A lo largo del texto, se analizan sus bases teóricas, su nivel de complejidad computacional, destacando la clasificación NP-Difícil o NP-Completo presente en varios de ellos, y las metodologías estructuradas para abordarlos, evitando la ineficiencia de la búsqueda exhaustiva.

PROBLEMA DE PROGRAMACIÓN DE TRABAJOS (JSSP)

Descripción del problema

El problema de Programación de trabajos (JSSP) es un problema de optimización ampliamente estudiado con diversas aplicaciones industriales. El objetivo principal es definir cómo minimizar el tiempo de fabricación necesario al asignar recursos compartidos (máquinas) a lo largo del tiempo para lograr completar actividades competitivas (trabajos) [1].

Este modelo se clasifica como un problema de optimización combinatoria que, además de minimizar el tiempo de fabricación, también puede buscar reducir otros criterios como la tardanza o el tiempo de flujo. Computacionalmente, pertenece a la categoría NP-Hard (o NP-Difícil), que son clases de complejidad computacional que miden qué tan difícil es resolver un problema usando una computadora. Como entrada, recibe un conjunto finito *J* de trabajos y un conjunto finito *M* de máquinas. El tiempo de procesamiento del trabajo *j* en la máquina *m* se define como un número entero no negativo, y se busca encontrar la secuencia óptima de *J* en *M* para minimizar el tiempo total de finalización [3].

Para que este modelo teórico funcione y se le pueden aplicar métodos de resolución, es necesario establecer las siguientes suposiciones [3]:

1. El número de operaciones para cada trabajo debe ser finito.
2. Se define el tiempo de procesamiento para cada operación en una máquina particular.
3. Existe una secuencia predefinida de operaciones que deben mantenerse para completar cada trabajo.
4. Los plazos de entrega de los productos deben ser indefinidos.
5. No hay costo de instalación ni costo de tardanza.
6. Una máquina solo puede procesar un trabajo a la vez.
7. Cada trabajo visita cada máquina solo una vez.
8. Ninguna máquina puede realizar más de un tipo de tarea.
9. El sistema no se puede interrumpir hasta que finalice cada operación de cada trabajo.
10. Ninguna máquina puede detener un trabajo y comenzar otro, antes de terminar el anterior.

A pesar de su gran utilidad industrial, la formulación teórica de este problema tiene una limitación importante: no maneja la incertidumbre. Al basarse en factores fijos y supuestos simplificadores, el modelo suele ignorar imprevistos del mundo real, como las averías de las máquinas o la escasez de materiales, lo que puede resultar en soluciones deficientes cuando ocurren emergencias [2].

Aun con estas limitaciones, la resolución de este modelo ayuda a las organizaciones a ahorrar tiempo y recursos valiosos en diversas áreas. En la industria manufacturera se utiliza para “fabricación flexible”, donde se optimiza la asignación de trabajos que requieren capacidades o equipos muy específicos. En la industria tecnológica, se aplica a la programación de tareas multiprocesador, en este caso, las instrucciones informáticas actúan como “trabajos” y los procesadores como “máquinas”, buscando maximizar la eficiencia de la computadora. Asimismo, con el avance de la automatización, el modelo se utiliza para optimizar los movimientos y secuencias de transporte de sistemas robóticos, logrando la mayor productividad posible en la planta [3].

REPRESENTACIÓN

Para llevar a cabo la representación visual y el análisis de este problema, una de las alternativas más efectivas y utilizadas es la elaboración de un diagrama de Gantt. Tomando como referencia un escenario práctico, se plantea un problema que cuenta con un tamaño de 3 x 3 (es decir, considerando un conjunto finito de tres trabajos J interactuando con un conjunto finito de tres máquinas M), donde es posible estructurar gráficamente una solución mediante la representación de intervalos de tiempo en forma de bloques (Figura 1). En esta estructuración, el eje vertical muestra las máquinas disponibles y el eje horizontal representa el tiempo transcurrido. Dentro de esta representación, se establece que el tiempo de funcionamiento dedicado por cada una de las máquinas presentará variaciones, ya que la asignación de estos lapsos dependerá directamente de las características y el tiempo de procesamiento del trabajo específico que se le encuentre asignado en ese instante. De esta manera, el modelo permite observar claramente la asignación de los recursos compartidos respetando las suposiciones del sistema para asegurar que no haya cruces ni superposiciones, garantizando así que una máquina procese únicamente un trabajo a la vez [3].

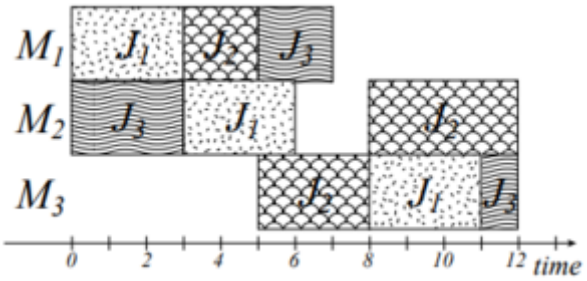


Figura 1: Representación en Diagrama de Gantt para un problema 3x3

Caso práctico: Ejemplo numérico

Para ilustrar la resolución del problema de programación de trabajos (JSSP), se presenta un caso de estudio aplicando el método de Ramificación y Acotamiento (Branch and Bound).
En este escenario, se requiere procesar cuatro trabajos distintos utilizando un único recurso compartido (una máquina). Cada trabajo cuenta con una duración de ejecución específica y una fecha límite de entrega dentro del cronograma general del proyecto. Los parámetros iniciales se definen en la siguiente tabla:

Trabajo	Duración (días)	Fecha de vencimiento
A	3	Día 5
B	5	Día 6
C	9	Día 16
D	7	Día 14

Tabla 1: Información del puesto

El objetivo de este método es determinar el orden en que se deben procesar los trabajos en el recurso compartido para minimizar el retraso total acumulado del proyecto. Existen n! soluciones factibles para un problema de programación de trabajos, por lo tanto, hay 4! = 24 soluciones factibles para este ejemplo. El día de finalización de este proyecto es la suma de todas las duraciones de los trabajos en una secuencia determinada. El retraso por trabajo se determina restando la fecha de vencimiento al día de finalización calculado. El retraso total del proyecto es la suma de todos los retrasos individuales. La siguiente tabla muestra un ejemplo para calcular el retraso de una secuencia particular, ABCD.

Trabajo	Día de finalización	Retraso
A	3	d_1 = 3-5 = -2 -> 0
B	3+5=8	*d2=8-6=2*
C	3+5+9=17	*d3 = 17-16 = 1*
D	3+5+9+7=24	*d4 = 24 - 14 = 10*

Tabla 2: Retraso de la secuencia (A, B, C, D)

Si se inicia con el trabajo A en el día 0, este concluirá en el día 3. Al terminar dos días antes de su fecha límite, no genera ningún retraso en el cronograma general. Para el trabajo B, se suman los 3 días utilizados previamente a sus 5 días de duración, resultando en un tiempo de finalización de 8 días. Esto produce un retraso de 2 días, ya que la fecha límite para el trabajo B era el día 6. Este proceso se repite para los trabajos C y D, lo que genera retrasos adicionales de 1 a 10 días, respectivamente. El retraso total de proyecto (T) es igual a la

suma del retraso acumulado de cada trabajo: T = *d1* + *d2* + *d3* + *d4* = 0 + 2 + 1 + 10 = 13 días.

Mediante el método de Branch and Bound, es posible aplicar este concepto para determinar la secuencia que produce el menor retraso de las 24 soluciones factibles. Primero, se definen las variables de decisión, donde xij = 1 si el trabajo j se coloca en la posición i, y xij = 0 en caso contrario. Las posiciones de los trabajos en la secuencia se indican con i = 1, 2, 3, 4, los trabajos se indican con j = A, B, C, D.

En esta etapa inicial, se desconoce la secuencia ideal de los trabajos, el retraso de cualquier trabajo individual y el retraso total del proyecto.

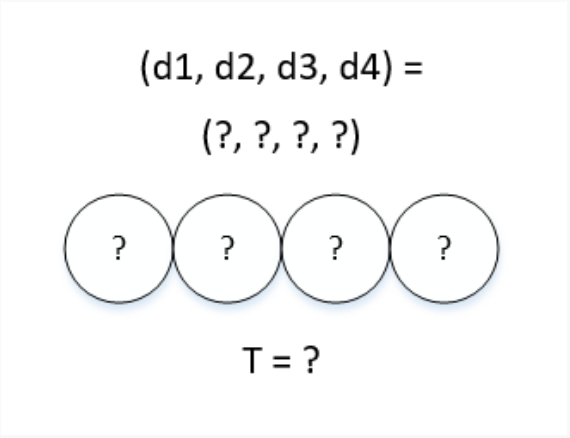


Figura 2. Paso 1: Estado inicial donde se desconoce la posición óptima para cada trabajo, el retraso individual y el retraso total del proyecto.

Se comienza evaluando la posición final de la secuencia. Siempre se conoce la fecha de finalización del último trabajo, ya que es la suma de todas las duraciones que deben completarse. En este ejemplo, se sabe que la tarea final concluirá el 24, como se observa en la Tabla 2. A continuación, se puede calcular el retraso que tendrá la tarea final restando su fecha de vencimiento de 24. Después de determinar qué trabajo producirá el menor retraso al estar en la última posición, se ramifica a partir de ese trabajo y se repite el proceso para el penúltimo trabajo de la secuencia. El trabajo A vence el día 5:24 - 5 = 19 de retraso. El trabajo B vence el día 6:24 - 6 = 18 de retraso. El trabajo C vence el día 16: 24 - 16 = 8 de retraso. El trabajo D vence el día 14:24 - 14 = 10 de retraso.

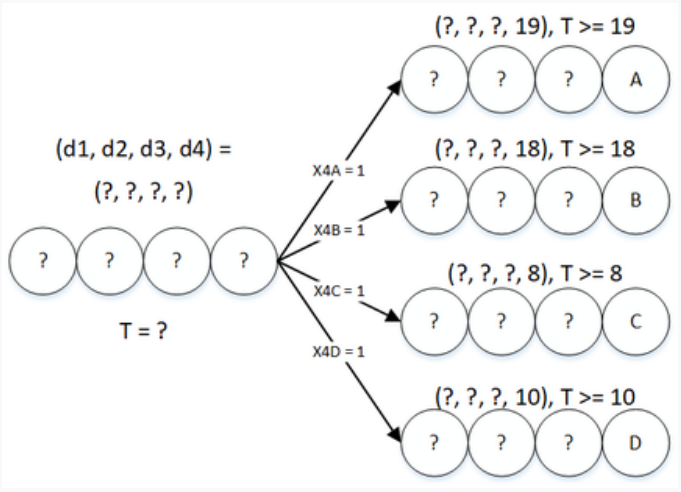


Figura 3. Paso 2: Determinación del menor retraso posible del proyecto asumiendo que cada trabajo se coloca en la última posición de la secuencia.

La Figura 3 muestra que el trabajo C en la posición final de la secuencia produce el menor retraso de proyecto posible T >= 8. Por lo tanto, se toma este camino y se repite el proceso de la tarea en la penúltima posición. Dado que el trabajo C se selecciona para ser el último y tiene una duración de 9 días, se sabe que el penúltimo trabajo se completará en 15 días. A partir de ahí se pueden volver a utilizar las fechas de vencimiento conocidas de cada tarea para determinar la ruta con el menor retraso acumulado. El trabajo A vence el día 5: 15 - 5 = 10 de retraso. El trabajo B vence el día 6: 15 - 6 = 9 de retraso. El trabajo D vence el día 14: 15 - 14 = 1 de retraso.

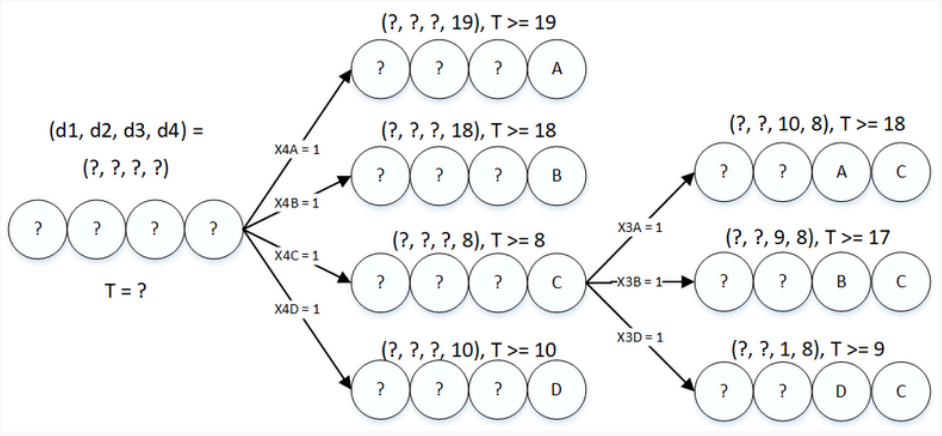


Figura 4. Paso 3: Evaluación desde el nodo C para determinar qué trabajo generará el menor retraso posible si se ejecuta en la penúltima posición. Se sabe que este trabajo se completará el día 15 (tiempo para completar el trabajo C restando del tiempo total para completar los trabajos: 24 - 9 = 15).

Al continuar determinando el límite inferior en cada nivel de decisión, eventualmente queda solo una opción para que un trabajo ocupe la primera posición. En este punto, se ha determinado la solución óptima, es decir, la secuencia que produce el menor retraso de

programación posible. La Figura 5 muestra el diagrama de Branch and Bound completo, ilustrando cómo se logra una solución óptima con la secuencia A-B-D-C y un retraso total de T = 11 días [3].

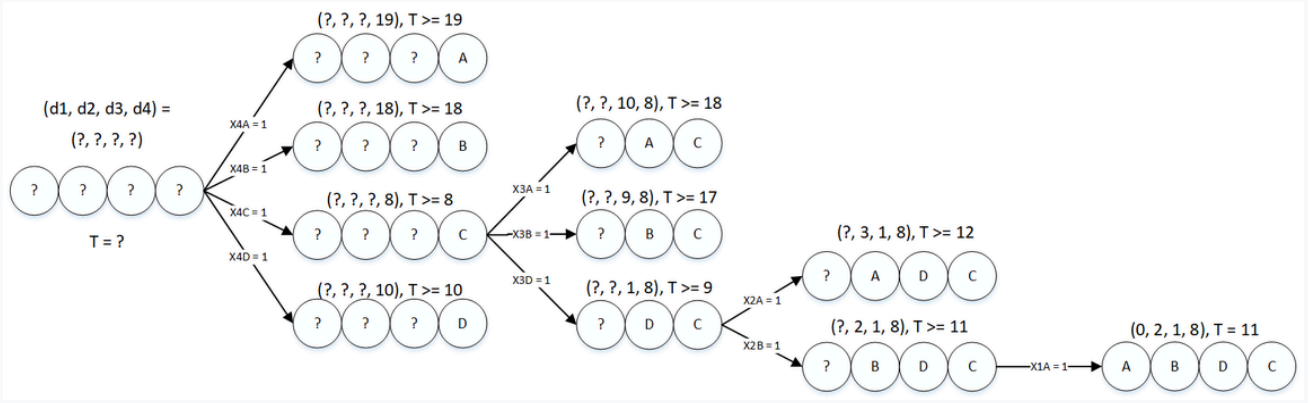


Figura 5: Pasos 4 y 5: Diagrama de flujo de Branch and Bound completo. Este método se utiliza para resolver un problema de programación de trabajos (JSSP) que coordina 4 trabajos que requieren el uso compartido de 1 máquina para completarse.

PROBLEMA DE LAS N REYNAS

El problema de las “N reinas” es un desafío clásico algorítmico en el que el objetivo principal es calcular de cuántas maneras diferentes se pueden colocar n reinas en un tablero de dimensiones n x n. La restricción fundamental es que ninguna de ellas deba amenazarse entre sí, es decir, ninguna reina puede ocupar la misma fila, columna o diagonal, evitando así que una pueda capturar a otra en un solo movimiento [4].

En esta formulación, la variable n representa la misma cantidad en ambos parámetros, lo que significa que el número de reinas debe ser exactamente igual al número de filas y columnas del tablero (cuyo caso particular más conocido es el de colocar 8 reinas en un tablero estándar de 8 x 8) [4].

Una característica notable de este problema es que la cantidad de soluciones válidas no escala de forma predecible en relación con el tamaño de n. Por ejemplo, para n = 5 existen 10 soluciones posibles, para n = 6,la cantidad se reduce a 4, mientras que para n = 7, el número de soluciones asciende a 40 [4]. En el caso específico de las 8 reinas, este cuenta con 92 soluciones distintas, pero si consideramos que las soluciones que difieren solo por rotaciones o reflexiones son iguales, entonces solo cuenta con 12 soluciones únicas [5]. Este comportamiento lo diferencia significativamente de variantes algorítmicas más simples, como el problema equivalente utilizando torres, donde el número de soluciones se calcula mediante la función factorial (n!) y presenta un crecimiento constante a medida que el valor de n aumenta [4]. Una particularidad notable de esta secuencia es que, cabe destacar que

solamente los tableros de n = 2 y n = 3 carecen por completo de soluciones. Para los primeros valores de n tenemos:

n	1	2	3	4	5	6	7	8	9	10	11	12
Únicas	1	0	0	1	2	1	6	12	46	92	341	1,787
Distintas	1	0	0	2	10	4	40	92	352	724	2,680	14,200

Representación

Para llevar a cabo la representación y el modelado de este problema de manera eficiente, el enfoque principal aprovecha una restricción matemática clave. Partiendo del hecho de que la cantidad total de reinas (n) es exactamente igual al número de filas y al número de columnas disponibles en el tablero, se deduce lógicamente que, dentro de cualquier solución que se considere válida, existirá exactamente una sola reina posicionada en cada fila. Si se presentara el caso de tener una fila completamente vacía, esto obligaría forzosamente a colocar dos o más reinas en alguna de las filas restantes, lo cual provocaría que se amenazaran mutuamente, invalidando el resultado.

Por lo tanto, el modelo descarta la representación tradicional que consistirá en verificar cada casilla individual del tablero de manera exhaustiva. Dicho enfoque ineficiente obligaría a procesar hasta $(n\,x\,n)^n$ combinaciones posibles, lo que además traería como consecuencia la evaluación de posiciones repetitivas en lugar de combinaciones únicas, generando un gran número de soluciones duplicadas. Para solucionar esto, el modelo se simplifica estructurando el problema mediante la asignación previa de cada una de las n reinas a cada una de las n filas correspondientes (asignando de manera efectiva la reina 1 a la fila 1, la reina 2 a la fila 2 y así sucesivamente). De esta forma la representación del problema se centra exclusivamente en buscar combinaciones de columnas para cada fila ya ocupada. Esta estrategia logra reducir drásticamente el espacio de búsqueda a un máximo de n^n combinaciones únicas, garantizando desde su planteamiento la exclusión de estados equivalentes [4].

CASO PRÁCTICO: IMPLEMENTACIÓN DEL ALGORITMO

Planteamiento del problema

En el ajedrez es posible colocar ocho reinas en el tablero de tal forma que ninguna reina pueda ser atacada por otra. El objetivo de este modelo es desarrollar un programa que

determine todas las posibles posiciones en que se pueden acomodar las reinas, dada la posición inicial de una de ellas.

Estructura de entrada

Para este escenario, la primera línea de la entrada contiene la cantidad de casos, seguida por un espacio en blanco. Cada caso consiste en dos números separados por un espacio, los cuales representan la posición inicial de una de las ocho reinas. Debido a que siempre se proporciona una posición permitida, se omite la validación de la entrada.

Para mantener un orden, se asume que la esquina superior izquierda del tablero ocupa la posición (1,1). Las filas se numeran en horizontal (siendo la superior la fila 1), y las columnas en vertical (iniciando desde la izquierda con la columna 1). Cualquier referencia a una casilla se hace primero por fila y después por columna, por lo tanto, la coordenada (4,6) se refiere a la fila 4, columna 6 (Figura 2)

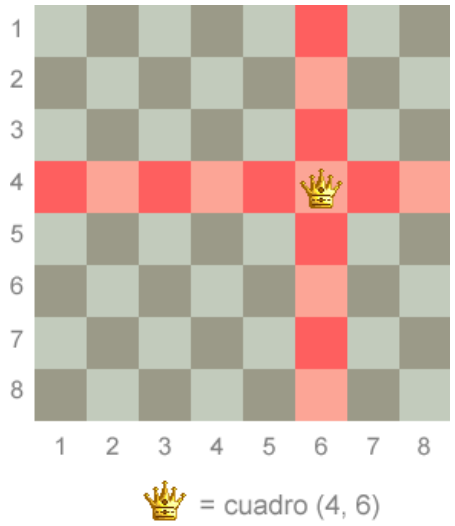


Figura 2. Ejemplo de ubicación en el tablero utilizando la lectura de coordenadas por fila y columna para la posición (4,6) [4].

Estructura de salida

Por cada caso en la entrada, se genera una línea con la solución a la salida. Cada solución se numera secuencialmente de 1 a n y contiene 8 número en una sola línea.

Cada uno de los 8 números representa la coordenada de la fila de la solución. La coordenada de la columna está dada por el orden en que los números impresos: el primer número representa la fila en la que está la reina de la primera columna, el segundo número es la fila de la reina en la segunda columna, así sucesivamente.

Por último, se incluyen dos líneas de encabezado, las soluciones se imprimen en orden numérico, y se deja una línea en blanco entre cada caso.

Solución

Se sabe que solo existen 92 soluciones, por lo que se generan de manera anticipada. Posteriormente, en cada caso se imprimen aquellas que contengan una reina en la casilla solicitada.

Para encontrar las soluciones, se coloca una reina por cada columna, revisando que no entre en conflicto con las reinas anteriores. Si la reina actual no se puede acomodar sin atacar a otra, o si ya se colocaron las ocho piezas, se retrocede y se cambia la posición de la casilla anterior. Cada vez que se logran acomodar todas, se guarda la solución.

Implementación del algoritmo

A continuación, se muestra el código del algoritmo escrito en el lenguaje Pascal (Figura 3). En este programa se pueden ver las variables utilizadas, cómo se inicializan los datos y la función recursiva que se encarga de revisar las posiciones para encontrar las 92 soluciones antes de leer lo que pide el usuario.

```
01:  Var
02:     i,j,k,m,n,s,t: longint;
03:     v,r: array [1..8] of longint;
04:     di,dd: array [1..15] of longint;
05:     sol: array [1..92,1..8] of longint;
06:
07:  Procedure Reinas(h: longint);
08:            var d,i,j,k: longint;
09:            begin
10:            for j:= 1 to 8 do
11:                begin
12:                d:= j+h-1; i:= 8+j-h;
13:                if (v[j]=0) and (di[i]=0) and (dd[d]=0) then
14:                   begin
15:                   v[j]:= h; di[i]:= h; dd[d]:= h; r[h]:= j;
16:                   if (h= 8) then begin
17:                                  Inc(s);
18:                                  for k:= 1 to 8 do
19:                                      sol[s,k]:= r[k];
20:                                  end
21:                              else Reinas(h+1);
22:                   v[j]:= 0; di[i]:= 0; dd[d]:= 0;
23:                   end;
24:                end;
25:            end;
26:
27:  Begin
28:       s:= 0;
29:       fillchar(v,sizeof(v),0);
30:       fillchar(di,sizeof(di),0);
31:       fillchar(dd,sizeof(dd),0);
32:       Reinas(1);
33:
34:       readln(input,t);
35:       for k:= 1 to t do
36:           begin
37:           readln(input);
38:           readln(input,m,n);
39:           writeln(output,'SOLN       COLUMN');
40:           writeln(output,' #      1 2 3 4 5 6 7 8');
41:           writeln(output);
42:           s:= 0;
43:           for i:= 1 to 92 do
44:               if (sol[i,n]= m) then
45:                  begin
46:                  Inc(s);
47:                  write(output,s:2,'      ');
48:                  for j:= 1 to 7 do
49:                      write(sol[i,j],' ');
50:                  writeln(output,sol[i,8]);
51:                  end;
52:           if (k <> t) then writeln(output);
53:           end;
54:  End.
```

Figura 3. Código fuente de la solución al problema de las N reinas implementado en Pascal.

En las primeras cinco líneas se declaran las variables de control: *i, j, k* para los ciclos: *s* para llevar la cuenta de las soluciones, *m* y *n* para la posición de la reina fija, y *t* para leer el número de casos. En lugar de usar una matriz normal para representar el tablero, se utilizan tres arreglos (v, di, dd) para saber rápido si las columnas diagonales están bajo ataque. El arreglo *r* guarda la solución actual y en *sol* se guardan todas las soluciones complejas

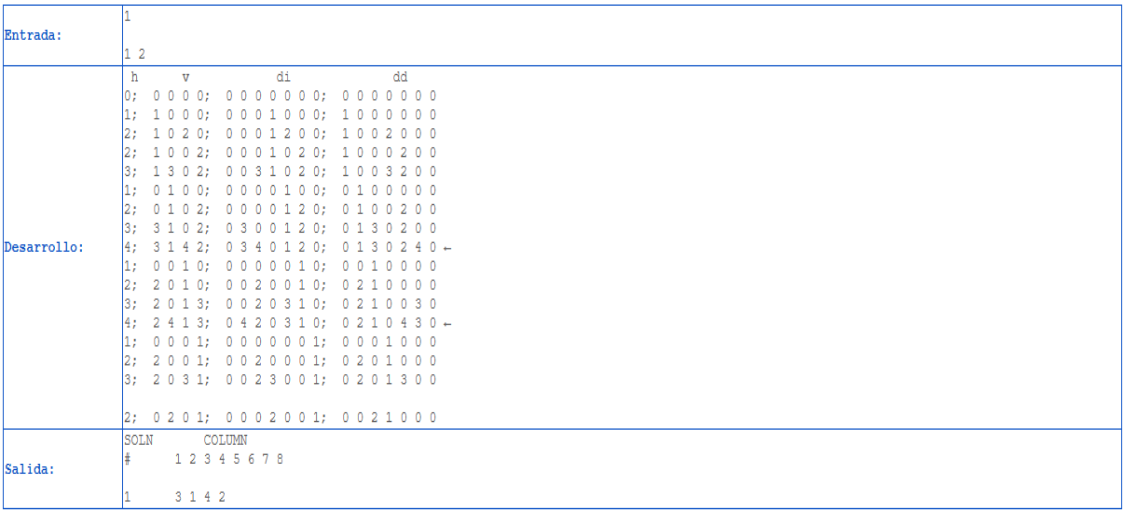
Las soluciones se generan en el procedimiento *reinas* (línea 7 a 25). Se utiliza el parámetro *h* para indicar en qué fila se pondrá la reina, lo que también sirve para saber cuántas reinas ya se han colocado. Para cada casilla de esa fila, se intenta poner a la reina revisando que no sea atacada por las anteriores (línea 13) usando *d* e *i* para las diagonales y *j* para las columnas. Si la casilla está libre, se anota a la reina en los arreglos y en la solución actual (línea 15).

Si ya se lograron colocar las 8 reinas, se guarda la solución (líneas 16 a 20), si todavía faltan, se pasa a la siguiente fila (línea 21). Al regresar de la recursividad, se quitan los valores que se habían asignado (línea 22) para intentar con otras posiciones. Al hacer esta revisión antes de acomodar a las reinas, los vértices visitados se reducen drásticamente a 8^8 a solo 1965.

Por último, entre las líneas 27 y 54 se encuentra la parte principal del código. Primero se inician las variables y se mandan a calcular las soluciones (línea 32). Luego se lee la cantidad de casos y la posición de la reina que ya está colocada. Después de imprimir los encabezados de formato, el programa busca e imprime únicamente las soluciones que tienen una reina en la casilla solicitada (*m, n*), separando cada caso con una línea en blanco al final.

Ejecución paso a paso (Caso ejemplo)

Para ver el funcionamiento del programa, se presenta un caso de prueba (Figura 4). La tabla detalla qué datos entran, cómo cambian las variables (h,v,di,dd) paso a paso y el resultado de salida. Este ejemplo utiliza 4 reinas para poder mostrar todo el procedimiento de forma más clara y breve.



Entrada:

```
1

1 2
```

Desarrollo:

```
 h       v            di              dd
0;  0 0 0 0;  0 0 0 0 0 0 0 0;  0 0 0 0 0 0 0 0
1;  1 0 0 0;  0 0 0 1 0 0 0 0;  1 0 0 0 0 0 0 0
2;  1 0 2 0;  0 0 0 1 2 0 0 0;  1 0 0 2 0 0 0 0
2;  1 0 0 2;  0 0 0 1 0 2 0 0;  1 0 0 0 2 0 0 0
3;  1 3 0 2;  0 0 3 1 0 2 0 0;  1 0 0 3 2 0 0 0
1;  0 1 0 0;  0 0 0 0 1 0 0 0;  0 1 0 0 0 0 0 0
2;  0 1 0 2;  0 0 0 0 1 2 0 0;  0 1 0 0 2 0 0 0
3;  3 1 0 2;  0 3 0 0 1 2 0 0;  0 1 3 0 2 0 0 0
4;  3 1 4 2;  0 3 4 0 1 2 0 0;  0 1 3 0 2 4 0 0 ←
1;  0 0 1 0;  0 0 0 0 0 1 0 0;  0 0 1 0 0 0 0 0
2;  2 0 1 0;  0 0 2 0 0 1 0 0;  0 2 1 0 0 0 0 0
3;  2 0 1 3;  0 0 2 0 3 1 0 0;  0 2 1 0 0 3 0 0
4;  2 4 1 3;  0 4 2 0 3 1 0 0;  0 2 1 0 4 3 0 0 ←
1;  0 0 0 1;  0 0 0 0 0 0 0 1;  0 0 0 1 0 0 0 0
2;  2 0 0 1;  0 0 2 0 0 0 0 1;  0 2 0 1 0 0 0 0
3;  2 0 3 1;  0 0 2 3 0 0 0 1;  0 2 0 1 3 0 0 0

2;  0 2 0 1;  0 0 0 2 0 0 0 1;  0 0 2 1 0 0 0 0
```

Salida:

```
SOLN       COLUMN
#       1 2 3 4 5 6 7 8

1       3 1 4 2
```

Figura 4. Ejemplo de ejecución del algoritmo mostrando la entrada, el desarrollo de las variables y la salida para un caso de 4 reinas.

Árbol de expansión mínima (MST).

Definición Conceptual

En teoría de grafos, un **árbol de expansión** (o árbol recubridor) de un grafo conexo y no dirigido es un subgrafo que cumple con dos características esenciales:

- **Es conexo y abarca la totalidad de los nodos:** Existe un camino único que conecta cualquier par de vértices del grafo original.
- **Es acíclico:** No contiene bucles cerrados ni circuitos que regresen al punto de partida.

Cuando las aristas del grafo poseen valores numéricos asociados —como costos, distancias o tiempos— se habla de un grafo ponderado. En este escenario surge el **Árbol de Expansión Mínima** (frecuentemente abreviado como **MST**, por sus siglas en inglés *Minimum Spanning Tree*), el cual se define como aquel árbol de expansión cuya **suma total del peso de sus aristas es la menor posible** entre todas las combinaciones que logran mantener interconectados a todos los vértices.

En caso de que el grafo original no sea conexo (es decir, que se encuentre fragmentado en varias islas o componentes aisladas que no pueden comunicarse entre sí), no es posible trazar un único árbol que alcance a todos los nodos. En esta situación, el concepto se generaliza a un **bosque de expansión mínima**, el cual consiste en la unión de los árboles de expansión mínima calculados de forma independiente para cada una de las componentes conexas.

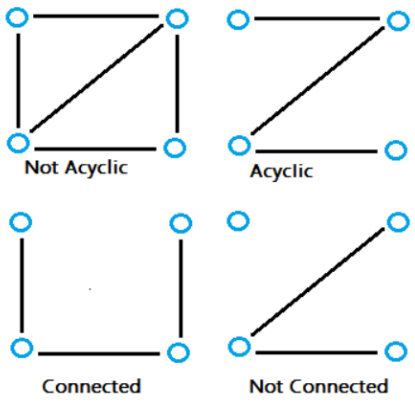
Motivación práctica

El objetivo de un MST es eliminar la redundancia y optimizar recursos al interconectar un sistema. Un caso clásico es el de una empresa de telecomunicaciones que necesita tender cableado para dar servicio a varios vecindarios: cada intersección o vivienda representa un vértice, y cada ruta viable para instalar el cable representa una arista con un costo de tendido determinado. Al calcular el árbol de expansión mínima, la compañía garantiza que todas las casas queden comunicadas entre sí utilizando la menor cantidad posible de cable y minimizando el gasto total de la obra.

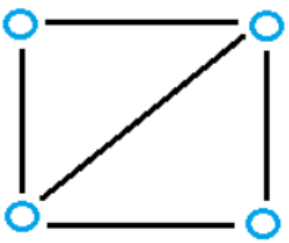
A continuación, se presentan ejemplos visuales y numéricos que ilustran cómo se identifican estas estructuras dentro de un grafo.

Un **árbol** tiene un camino que une dos vértices cualesquiera. Un **árbol de expansión** de un gráfico es un árbol que:

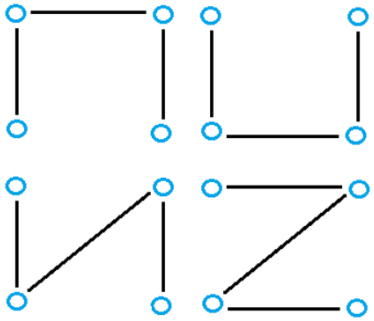
- Contiene todos los vértices del gráfico original.
- Alcanza (abarca) todos los vértices.
- El gráfico no tiene ningún nodo que vuelva a sí mismo.



Incluso los gráficos más simples pueden contener muchos árboles de expansión. Por ejemplo, el siguiente gráfico:



tiene muchas posibilidades para los árboles de expansión, que incluyen:



Caso de uso y analogía práctica

Un ejemplo clásico para entender este problema se presenta en las empresas de telecomunicaciones o de servicios básicos al instalar infraestructura (como tendido de cable coaxial o fibra óptica) en una nueva zona residencial.

- Los puntos de entrega o casas se modelan matemáticamente como **vértices**.
- Las calles, ductos o caminos permitidos donde la empresa tiene autorización para enterrar el cable representan las **aristas**.
- El costo asociado a cada tramo (que depende de la distancia, la dificultad del suelo o la profundidad requerida de zanja) corresponde al **peso de la arista**.

En este tipo de problemas, el costo no necesariamente tiene que cumplir con reglas geométricas habituales como la desigualdad del triángulo; el peso simplemente mide dinero, tiempo o esfuerzo. Un árbol de expansión en este grafo representará cualquier diseño de red que comunique a todas las casas sin generar circuitos redundantes. Por lo tanto, el árbol de expansión mínima representará la alternativa de diseño más económica posible para la instalación.

Propiedades Matemáticas y Teóricas

Cardinalidad y multiplicidad de soluciones

- **Número de aristas:** En cualquier grafo conexo con n vértices, todo árbol de expansión contendrá exactamente $n - 1$ aristas.
- **Existencia de múltiples árboles mínimos:** Un grafo puede tener más de una solución óptima válida con exactamente el mismo costo total. Por ejemplo, en el

caso extremo donde todas las aristas del grafo posean exactamente el mismo peso, cualquier árbol de expansión que se construya será automáticamente un árbol de expansión mínima.

Condición de unicidad

Si en el grafo original **todos los pesos de las aristas son estrictamente diferentes**, entonces el MST resultante será **único**. Esta condición es común en problemas reales (como el del cableado), ya que rara vez dos tramos de construcción tienen exactamente el mismo costo al centavo. Si no todos los pesos son diferentes pero hay empates, lo único que se garantiza como único es el conjunto multiconjunto de valores de los pesos que forman el MST, aunque las aristas específicas varíen.

Demostración de unicidad (por contradicción):

- Supongamos que existen dos árboles de expansión mínima diferentes, A y B. Como ambos cubren los mismos vértices pero no son idénticos, debe existir al menos una arista que pertenece a uno pero no al otro. De entre todas esas aristas que no comparten, elegimos la arista e_1 de menor peso (la cual es única porque definimos que ningún peso se repite). Sin pérdida de generalidad, asumimos que e_1 está en A y no en B.
- Si agregamos e_1 al árbol B, se crea necesariamente un ciclo C que contiene a e_1. Dado que el árbol A no tiene ciclos, el ciclo C debe contener por fuerza otra arista e_2 que no pertenece a A.
- Como habíamos seleccionado e_1 como la arista con el menor peso entre todas las que estaban en un solo árbol, se deduce que el peso de e_2 es forzosamente mayor que el de e_1. Si reemplazamos e_2 por e_1 dentro del árbol B, el ciclo se rompe y obtenemos un nuevo árbol con un costo total menor al de B. Esto contradice la premisa original de que B era un árbol mínimo; por lo tanto, no pueden existir dos árboles mínimos distintos.

Subgrafo de costo mínimo

Cuando todos los pesos de las aristas son positivos, el MST es formalmente el subgrafo con menor costo total capaz de conectar todos los vértices. Cualquier otro subgrafo conexo que contenga ciclos tendrá aristas sobrantes sumando peso extra innecesariamente.

Propiedad del ciclo

Para cualquier ciclo simple C presente en el grafo: si una arista e dentro de C tiene un peso estrictamente mayor que el de todas las demás aristas que componen dicho ciclo, esa arista **no puede formar parte de ningún MST**.

- **Demostración:** Si asumimos que e formara parte de un árbol mínimo T_1, al quitarla romperíamos T_1 en dos subárboles disjuntos. Como el resto del ciclo C conecta los mismos extremos que unía e, tiene que existir al menos otra arista f perteneciente a C que conecte ambos subárboles. Si conectamos los subárboles usando f en lugar de e, formamos un nuevo árbol T_2. Como el peso de f es menor que el de e, el peso total de T_2 es menor que el de T_1, lo cual invalida que T_1 fuera mínimo.

Propiedad del corte

Un *corte* $(S, V - S)$ es una partición de los vértices del grafo en dos grupos disjuntos. Si consideramos todas las aristas que cruzan de un grupo a otro a través de ese corte: si una arista e tiene un peso estrictamente menor que el resto de las aristas que cruzan el corte, entonces e **pertenece a todos los árboles de expansión mínima posibles**.

- **Demostración:** Supongamos un MST T que no incluya a la arista e. Si añadimos e a T, formaremos un ciclo que cruza el corte en e y debe volver a cruzarlo en otra arista e'. Si retiramos e' e introducimos e, obtenemos un nuevo árbol conexo $T' = (T \setminus \{e'\}) \cup \{e\}$. Como el peso de e es menor que el de e', el costo de T' es menor que el de T, lo cual contradice que T fuera mínimo. De manera equivalente, si existen múltiples aristas que empatan en el menor peso del corte, cada una de ellas formará parte de al menos un MST válido.

Arista de costo mínimo global y contracción

- **Arista mínima global:** Si el grafo tiene una arista con el menor peso de todo el grafo y ese peso es único, dicha arista formará parte de cualquier MST (basta aplicar la propiedad de corte tomando como conjunto un solo vértice extremo de esa arista).

- **Contracción:** Si ya sabemos con certeza que un conjunto de aristas T forma parte del MST, podemos «contraer» esas aristas y fusionar sus vértices en un solo

supernodo. Si resolvemos el MST del nuevo grafo reducido y le sumamos las aristas contraídas de T, el resultado sigue siendo el MST correcto del grafo original.

Algoritmos de Resolución

En los análisis de complejidad siguientes, n representa la cantidad de vértices (|V|) y m la cantidad de aristas (|E|).

Algoritmos Clásicos (Estrategias Voraces)

Conforme un grafo crece en cantidad de vértices y aristas, el número de árboles de expansión posibles se dispara exponencialmente, alcanzando con facilidad millones o miles de millones de combinaciones. Intentar encontrar el árbol de expansión mínima mediante fuerza bruta (probando cada subgrafo uno por uno) resulta computacionalmente inviable. A esta complejidad combinatoria se suma que las aristas suelen tener pesos heterogéneos; por ejemplo, dos tramos con la misma distancia física de 5 metros podrían tener costos distintos (uno con peso 5 y otro con peso 7) debido a factores como el tipo de terreno, la dificultad de instalación o los materiales requeridos.

Para resolver este desafío sin recurrir a una búsqueda exhaustiva, los métodos tradicionales emplean **estrategias voraces** (*greedy*). Estos algoritmos construyen la solución paso a paso, tomando en cada iteración la mejor decisión local disponible (la arista de menor peso que cumpla con las reglas) con la garantía matemática de alcanzar el óptimo global.

Gracias a este enfoque, el cálculo del MST no sufre de una explosión combinatoria inmanejable, sino que se ejecuta en **tiempo polinomial**. En términos de teoría de la complejidad, la tarea de calcular el árbol pertenece a la clase **FP** (problemas de funciones computables en tiempo polinomial), mientras que sus variantes de decisión asociadas —como determinar si existe un MST con peso total inferior a un valor dado o verificar si una arista específica forma parte de la solución óptima— se clasifican dentro de **P**.

Entre los métodos clásicos más importantes para resolver este problema destacan los algoritmos de **Kruskal**, **Prim** y **Borůvka**. Todos ellos garantizan encontrar la solución exacta de manera eficiente, sirviendo como la base teórica y computacional sobre la cual se implementan los sistemas de optimización y el software especializado en diseño de redes.

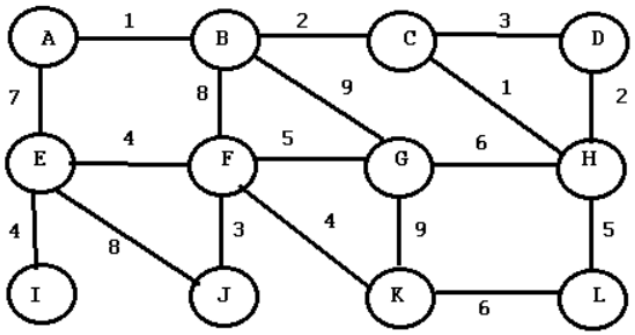
1. **Algoritmo de Borůvka (1926):** Fue históricamente el primer método desarrollado para este problema, creado por el matemático checo Otakar Borůvka con el fin de

diseñar de forma eficiente la red de distribución eléctrica de Moravia. Trabaja por etapas o rondas: en cada etapa, cada componente asociada elige de forma simultánea su arista incidente de menor peso. Luego, todas esas aristas se contraen formando un nuevo grafo para la siguiente ronda. Como en cada paso la cantidad de componentes se reduce al menos a la mitad, el algoritmo termina en $O(m \log n)$ pasos.

Ejemplo del algoritmo de Boruvka

Encuentre el árbol de expansión mínimo para el siguiente gráfico usando el Algoritmo de Buruvka:

Paso 1: Escriba una lista de componentes. Para este gráfico tenemos: A,B,C,D,E,F,G,H,I,J,K,L. Este paso es opcional pero le ayuda a realizar un seguimiento.

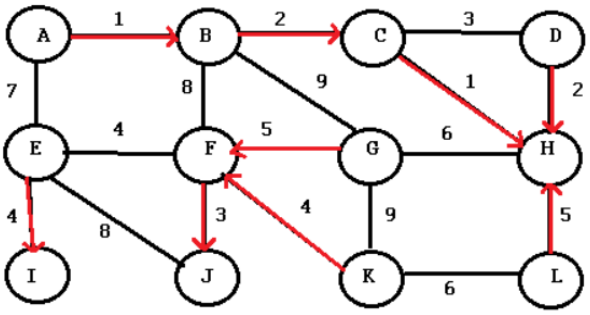


Paso 2: Resalte el borde saliente más barato para cada nodo en su lista. Por ejemplo, el nodo A tiene bordes salientes con pesos 1 y 7, por lo que resaltaremos 1. Continúe secuencialmente (para esta lista, vaya a B, luego a C…).

Directrices :

1. En esta etapa, solo resalte **un** borde más barato para cada nodo.
2. Para un empate (el nodo E tiene dos aristas con un peso de 4), asigne un peso menor a una de las aristas. Esto es arbitrario, pero debe permanecer constante durante todo el proceso. Para este ejemplo, haré que el borde de la izquierda tenga el peso más bajo.
3. Resalte siempre el borde más barato, *incluso si se ha resaltado antes* . **¡No** elija el borde con el siguiente peso más bajo! A efectos prácticos, esto significa que es

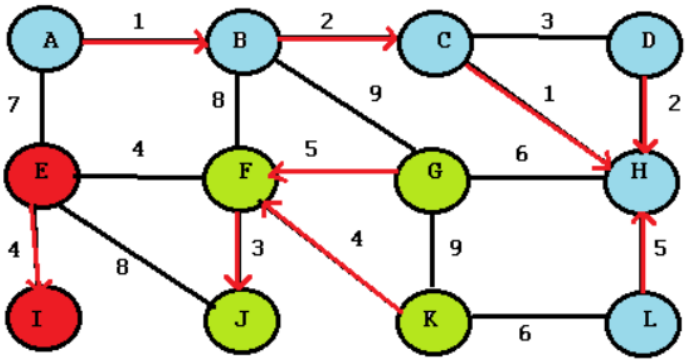
posible que no tenga que resaltar nada para algunos nodos, si ya se ha resaltado el peso más bajo.



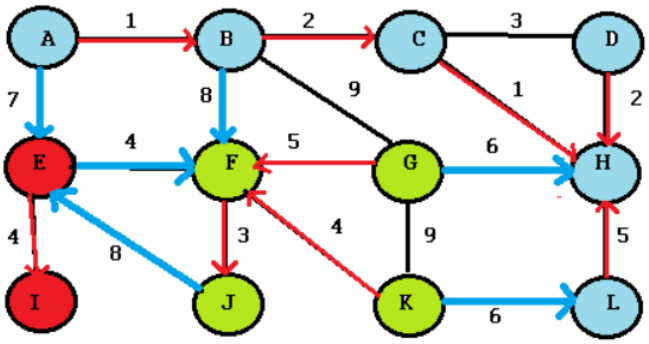
Paso 3: Resalte los grupos de árboles separados. Estos son los conjuntos de nodos conectados, a los que llamaremos *componentes* .

Paso 4: Repita el algoritmo para cada componente (cada conjunto de color diferente). Esta vez, para cada nodo, elija el borde más barato **fuera del componente** . Por ejemplo, ABCDHL es un componente (nodos azules). Para el nodo A, el borde más barato fuera del componente es el nodo 7 (porque el nodo 1 está conectado dentro del componente). Estoy resaltando en azul para mayor claridad: puedes usar el color que quieras.

Directrices :

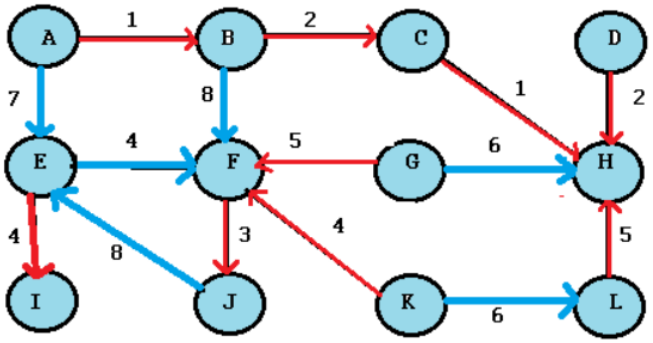


1. Asegúrese de elegir el borde más barato *fuera* del componente. Para este gráfico y esta iteración, esos bordes están coloreados en negro.
2. Como en la primera iteración, omita resaltar un borde si ya se ha resaltado esta vez.



Paso 5: si todos sus nodos están conectados en un solo componente, ya está. Si no, repita el paso 4.

El siguiente gráfico muestra el MST final. He borrado los bordes no utilizados (negros).

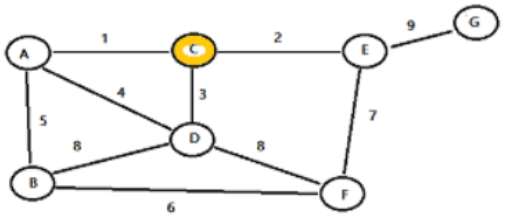


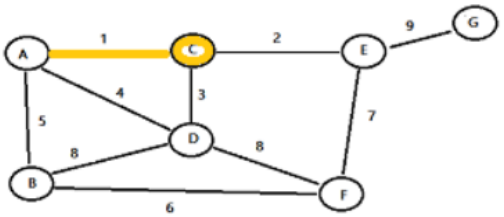
2. **Algoritmo de Prim (1930 / 1957 / 1959):** Propuesto originalmente por Vojtěch Jarník, redescubierto por Robert C. Prim y posteriormente por Edsger Dijkstra. Comienza seleccionando un vértice arbitrario como árbol inicial T. En cada iteración, busca la arista de menor peso (x, y) tal que x ya pertenezca al árbol y el nodo y aún no. Al seleccionarla, incorpora y y la arista al árbol. Por la propiedad del corte, cada elección es segura. Su tiempo de ejecución suele ser $O(m \log n)$ utilizando colas de prioridad binarias, o $O(m + n \log n)$ implementado con montículos de Fibonacci (*Fibonacci heaps*).

Cómo ejecutar el algoritmo de Prim

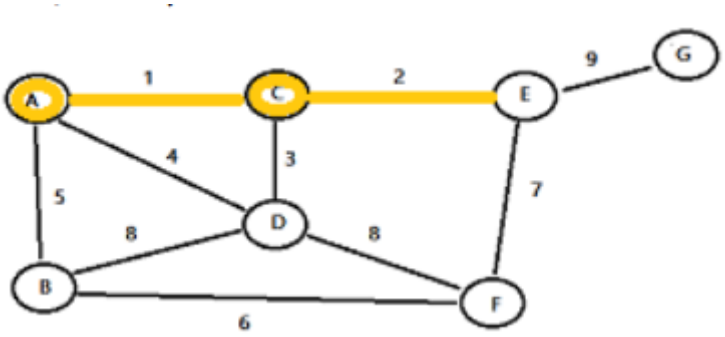
Paso 1: elige un nodo aleatorio y resáltalo. Para este ejemplo, elijo el nodo C. Paso 2: encuentre todos los bordes que van a los nodos no resaltados. Para este ejemplo, el nodo C tiene tres bordes con pesos 1, 2 y 3. Resalte el borde con el peso más bajo. Para este

ejemplo, eso es 1. Paso 3: Resalte el nodo que acaba de alcanzar (en este ejemplo, ese es el nodo A).





Paso 4: mire todos los nodos resaltados hasta ahora (en este ejemplo, son A y C). Resalte el borde con el peso más bajo (en este ejemplo, ese es el borde con peso 2). **Nota** : si tiene más de un borde con el mismo peso, elija uno al azar.



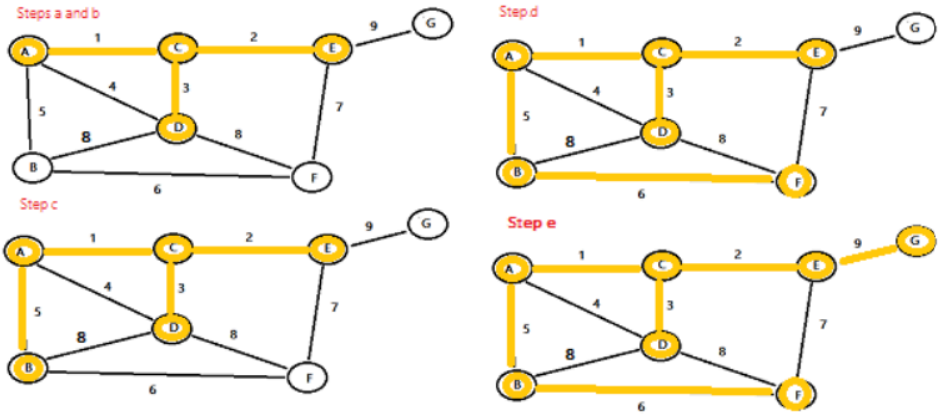
Paso 5: Resalte el nodo que acaba de alcanzar.

Paso 6: Resalte el borde con el peso más bajo. Elija entre todos los bordes que:

- Procede de todos los nodos resaltados.
- Llega a un nodo que aún no has resaltado

Paso 7: Repita los pasos 5 y 6 hasta que no tenga más nodos sin resaltar. Para este ejemplo en particular, los pasos específicos restantes son:

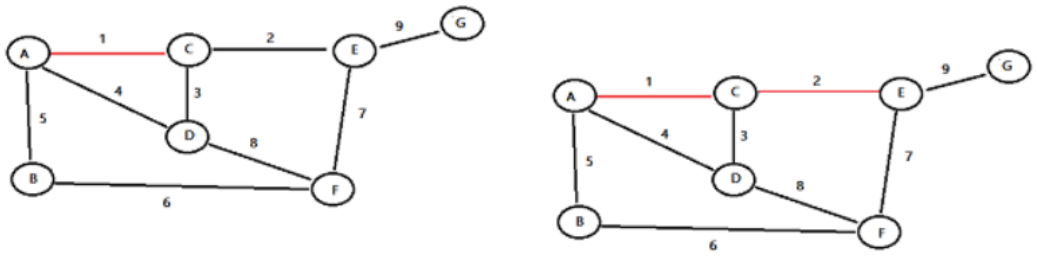
- una. Resalte el nodo E.
- b. Resalte el borde 3 y luego el nodo D.
- C. Resalte el borde 5 y luego el nodo B.
- d. Resalte el borde 6 y luego el nodo F.
- mi. Resalte el borde 9 y luego el nodo G.



3. **Algoritmo de Kruskal (1956):** Consiste en ordenar previamente todas las m aristas del grafo de menor a mayor costo. Posteriormente, las recorre en orden creciente y añade cada arista al conjunto solución siempre y cuando no forme un ciclo con las ya seleccionadas (para verificar esto rápidamente se utiliza la estructura de conjuntos disjuntos o *Union-Find* con compresión de caminos). Su complejidad temporal está dominada por el ordenamiento inicial: $O(m \log n)$.

Ejemplo del algoritmo de Kruskal

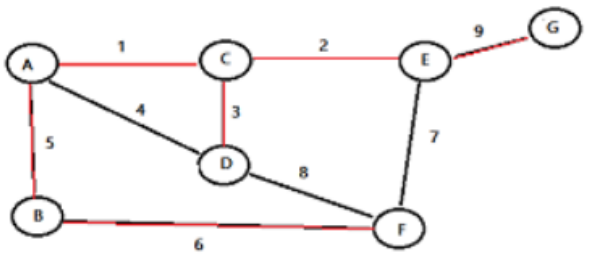
Encuentra el borde con el menor peso y resáltalo. Para este gráfico de ejemplo, he resaltado el borde superior (de A a C) en rojo. Tiene el peso más bajo (de 1): Encuentre el siguiente borde con el peso más bajo y resáltelo: Continúe seleccionando los bordes más bajos hasta **que todos los nodos** estén en el mismo árbol. **Notas** :



1. Si tiene más de un borde con el mismo peso, elija un borde con el peso más bajo.

2. Tenga cuidado de no completar un ciclo (enrutar un nodo de vuelta a sí mismo). Si su elección completa un ciclo, descarte su elección y pase al siguiente peso más grande.

El árbol de expansión mínimo terminado para este ejemplo se ve así:



4. **Algoritmo de Eliminación Inversa (*Reverse-Delete*):** Funciona de manera contraria a Kruskal. Comienza con el grafo completo y analiza las aristas en orden descendente (de la más pesada a la más liviana). En cada paso, elimina la arista de mayor peso a menos que al quitarla se desconecte el grafo. Su complejidad es de $O(m \log n(\log \log n)^3)$.

Algoritmos Avanzados y Casos Especiales

- **Algoritmos aleatorizados:** Karger, Klein y Tarjan (1995) desarrollaron un método aleatorizado (*randomized*) basado en comparar pares de aristas combinando la lógica de Borůvka con la eliminación inversa, logrando encontrar el MST en tiempo lineal esperado $O(m + n)$.
- **Algoritmo casi lineal de Chazelle:** Bernard Chazelle propuso el algoritmo determinista basado en comparaciones más rápido conocido, utilizando colas de prioridad aproximadas (*soft heaps*). Su complejidad es $O(m \cdot \alpha(m, n))$, donde α representa la inversa de la función de Ackermann. Como esta función crece de forma sumamente lenta (en situaciones prácticas nunca supera el valor de 4), se considera un rendimiento prácticamente lineal.
- **Grafos densos:** Si la densidad de aristas es alta ($m/n \geq \log \log \log n$), Fredman y Tarjan demostraron que se puede resolver de forma determinista en tiempo lineal estricto $O(m)$ mediante múltiples fases acotadas del algoritmo de Prim.
- **Pesos enteros:** Si los pesos no son números reales arbitrarios sino enteros representados en binario, existen algoritmos deterministas que calculan el árbol en tiempo $O(m + n)$.

- **Árboles de decisión y algoritmo óptimo:** Para grafos pequeños con pesos desconocidos, es posible construir un árbol de decisión binario evaluando comparaciones entre aristas. Seth Pettie y Vijaya Ramachandran diseñaron un algoritmo determinista demostrado matemáticamente como óptimo: aunque la fórmula exacta de su cota asintótica de tiempo aún no se conoce con exactitud, se demostró formalmente que ningún algoritmo basado en comparaciones puede superar la cota del árbol de decisión óptimo.
- **Procesamiento paralelo, distribuido y memoria secundaria:**
    - Con múltiples procesadores, el cálculo del MST se puede paralelizar para resolverse en tiempo $O(\log n)$.
    - En modelos distribuidos (donde cada nodo representa una computadora independiente que solo conoce sus propios enlaces vecinos), existen protocolos para calcular el MST sin necesidad de un nodo central.
    - Para grafos masivos que exceden la memoria RAM, los algoritmos de memoria externa (*external memory*) aplican ordenamiento en disco y contracción de grafos manteniendo tiempos solo de 2 a 5 veces más lentos que los métodos tradicionales en memoria principal.

Comportamiento en Grafos Completos

Alan M. Frieze y otros investigadores estudiaron el comportamiento probabilístico del MST sobre grafos completos de n vértices donde los pesos de las aristas son variables aleatorias continuas e idénticamente distribuidas en el intervalo $[0, 1]$.

Conforme n tiende a infinito, el valor esperado del peso total del árbol converge hacia un valor constante relacionado con la función zeta de Riemann, específicamente $\zeta(3) \approx 1.202$ (la constante de Apéry). Para grafos pequeños, los valores esperados exactos son:

Vértices (n)	Valor Esperado Exacto	Aproximación Decimal
2	$1/2$	0.5000
3	$3/4$	0.7500

4	$31/35$	0.8857
5	$893/924$	0.9665
6	$278/273$	1.0183
7	$30739/29172$	1.0537
8	$199462271/184848378$	1.0791
9	$126510063932/115228853025$	1.0979

Aplicaciones Prácticas

El MST no se limita únicamente al cableado o distribución de recursos; es una herramienta base en múltiples campos:

- **Ingeniería y redes de infraestructura:** Planificación de redes de cómputo (topologías de difusión sin bucles), redes eléctricas, redes de suministro de agua potable y trazado de vías férreas o carreteras.
- **Ciencia de datos y Bioinformática:** Análisis de agrupamiento (*clustering* jerárquico de enlace simple), agrupación taxonómica de especies y análisis de perfiles de expresión genética.
- **Visión por computadora y procesamiento gráfico:** Segmentación de regiones en imágenes, reconocimiento de escritura en expresiones matemáticas y extracción de contornos o rasgos curvilíneos.
- **Diseño de circuitos electrónicos:** Optimización de multiplicaciones constantes en el diseño de filtros digitales de respuesta al impulso finito (FIR).

- **Economía y finanzas:** Modelado de correlación entre activos bursátiles; calculando matrices de correlación entre acciones se construyen árboles de expansión mínima para visualizar la estructura del mercado financiero.
- **Subrutina de optimización:** Se utiliza como paso intermedio en algoritmos de aproximación para problemas NP-difíciles, como el algoritmo de Christofides para el Problema del Viajante de Comercio (TSP) y aproximaciones de corte multiterminal.

Variantes y Problemas Relacionados

- **Árbol de Steiner:** A diferencia del MST que debe conectar todos los nodos, en el árbol de Steiner se busca conectar un subconjunto específico de vértices y se permite introducir puntos o nodos intermedios artificiales (*puntos de Steiner*) para disminuir la longitud total. Este problema es clasificado como **NP-completo**.
- k**-MST:** Busca encontrar el árbol de expansión con el menor costo posible que conecte exactamente un subconjunto de k nodos dentro del grafo.

- **MST Euclidiano y Rectilíneo:** Casos especiales donde los vértices representan puntos en un plano o espacio geométrico y los pesos de las aristas corresponden directamente a su distancia euclidiana o distancia Manhattan (*rectilínea*).
- **Árbol de Expansión con Restricción de Grado:** Exige que ningún nodo esté conectado a más de un número determinado d de aristas dentro del árbol solución. Para $d = 2$, el problema equivale al Problema del Agente Viajero (TSP), por lo que en términos generales es **NP-duro**.
- **MST Capacitado (CMST):** Define un nodo raíz central y limita la cantidad máxima de nodos que puede contener cada subárbol conectado a él. Es un problema NP-difícil que suele resolverse mediante heurísticas como *Esau-Williams*.
- **Arborescencia (MST en grafos dirigidos):** En grafos con dirección, el problema busca un árbol dirigido con raíz y se resuelve mediante el algoritmo de Chu-Liu/Edmonds en tiempo $O(E + V \log V)$.
- **Árbol de Expansión Máxima:** Busca la conexión sin ciclos con el mayor costo acumulado posible. Se resuelve fácilmente multiplicando todos los pesos de las aristas por -1 y ejecutando cualquier algoritmo estándar de MST (Prim o Kruskal).
- **Árbol de Cuello de Botella Mínimo (MBST):** No busca minimizar la suma global de aristas, sino minimizar el peso de la arista individual más pesada (*cuello de botella*) dentro del árbol. Todo MST es por definición un MBST, aunque un MBST no necesariamente garantiza ser un MST en la suma total.

Problema del agente viajero (TSP)

Descripción del Problema (TSP)

Definición general: El Problema del Agente Viajero (conocido como TSP, por sus siglas en inglés *Traveling Salesman Problem*) o PAV (Problema del Agente Viajero) plantea la situación en la que una entidad (como una persona, agente viajero, vehículo, autobús o avión) debe visitar un conjunto de nodos (ciudades, localidades, tiendas o empresas).

Condiciones y objetivo: Para resolverlo se establecen dos reglas esenciales:

    - Debe visitar cada nodo sin repetirlo (pasar exactamente una sola vez por cada uno).
    - Debe regresar obligatoriamente al punto o ciudad de partida. El objetivo principal es hallar la ruta o circuito completo que tenga el **menor costo total posible**.

- **Origen e historia:** La primera solución documentada data de 1954, publicada por George Dantzig, Ray Fulkerson y Selmer Johnson en su trabajo *"Solutions of a large scale traveling salesman problem"*, donde resolvieron una instancia de 49 ciudades bajo estas premisas.
- **Naturaleza y complejidad combinatoria:**
    - Está catalogado como un **Problema de Optimización Combinatoria**.
    - Su espacio de soluciones crece de forma exponencial conforme aumenta el número de nodos. Por ejemplo, para 3 nodos existen 6 posibles rutas; para 4 nodos sube a 12; para 10 nodos son 90; para 100 nodos alcanza 9,900 combinaciones teóricas; y en problemas como el de las 16 ciudades de la Odisea de Ulises (Grötschel y Padberg, 1993), existen más de 653 mil millones de rutas posibles ($653,837'184,000$).
    - Computacionalmente está clasificado como un problema **NP-Completo**, lo que significa que no se puede asegurar encontrar la solución óptima absoluta en un tiempo de cálculo razonable para instancias grandes. Por ello, se recurre frecuentemente a métodos heurísticos para obtener soluciones aproximadas de buena calidad en tiempos reducidos.

Representación del Problema

El documento describe la representación del TSP tanto a nivel estructural/gráfico como algorítmico y de variables:

- **Representación mediante Grafos:**

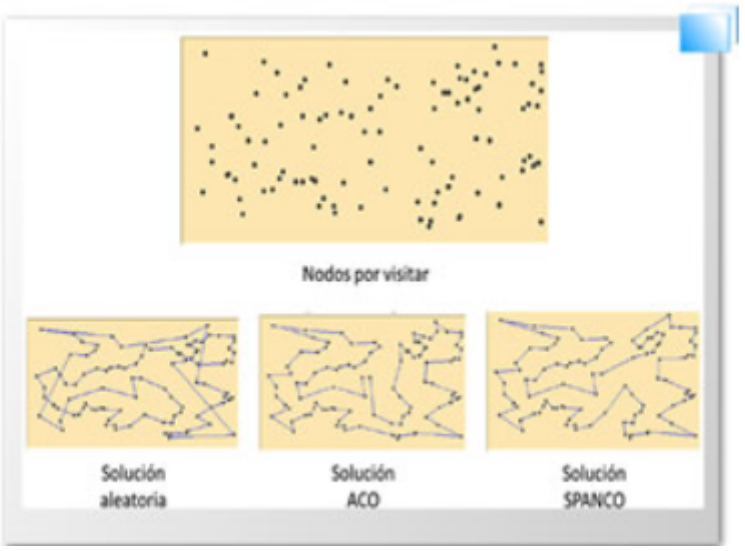
- **Nodos (Vértices):** Representan los puntos de control, lugares físicos o entidades que se deben visitar (ciudades, semáforos, domicilios, herramientas, componentes o puntos de soldadura).
- **Aristas (Enlaces):** Representan las conexiones o caminos viables que la entidad puede recorrer para trasladarse de un nodo a otro.
- **Pesos o Costos ($costo_{ij}$):** Cada arista que conecta a una ciudad i con una ciudad j tiene asignado un costo de traslado. Este costo no solo se mide en distancia métrica (metros, kilómetros, etc.), sino también en tiempo (horas, minutos) o en recursos consumidos (dinero, gasto de energía, desgaste de piezas, uso de combustible).

- **Representación del Algoritmo Base (Lógica del recorrido):**
  De acuerdo con el texto (citando a Öncan et al., 2009), el flujo del algoritmo base para representar y procesar el recorrido sigue esta estructura:
  - Definir el número total de nodos, su posición espacial y el costo asociado a cada arista (i, j) entre el nodo i y el nodo j.
  - Seleccionar un nodo de inicio i.
  - Ejecutar de forma iterativa (*Hacer*):
    - Verificar si el nodo más cercano no ha sido visitado aún.
    - Si no ha sido visitado, trasladarse al nodo j.
    - Actualizar el registro o lista de nodos visitados.
    - Acumular el costo en la variable total: $Costo_total = Costo_total + costo_{ij}$.
    - Actualizar la posición actual haciendo que el nodo i pase a ser el nodo j.
  - Repetir el ciclo hasta haber visitado todos los nodos del sistema.
- **Representación en Técnicas de Inteligencia Artificial:**
  El documento detalla que también puede modelarse según la técnica empleada:
  - En **Algoritmos Genéticos**, se representa mediante cromosomas donde cada «gen» es una variable o una ciudad, y la cadena de genes define el orden secuencial de visita.
  - En **Redes Neuronales**, cada neurona modela un nodo a visitar y las conexiones o sinapsis representan las relaciones y el vector de costos entre ellos.
  - En **Colonia de Hormigas (ACO)**, el problema se modela como un circuito cerrado donde el punto de partida es igual al punto de llegada y las entidades recorren los nodos depositando soluciones sin repetir ciudades.

Simulación de TCP.

En Fuentes-Penna and González-Ramírez (2013) se presenta una simulación del Problema del Agente Viajero (TSP por sus siglas en inglés), donde se tienen 100 ciudades que deberán ser visitadas en el menor tiempo y con el menor costo de traslado. Se asigna de manera aleatoria la distancia y el costo entre los nodos con un máximo de 10 nodos colindantes. El Algoritmo propuesto se denomina Scheduling Project Ant Colony Optimization (SPANCO) y se comparan los resultados con una solución aleatoria y con el algoritmo basado en Optimización de Colonia de Hormigas (ACO por sus siglas en inglés). En este artículo se muestra que el algoritmo SPANCO reduce el costo total y la distancia total del recorrido en un rango del 3% al 8%. La simulación se relaciona con el software AIMMS para modelado matemático y de optimización (AIMMS, 2013). Las áreas de aplicación de AIMMS son:

- Planificación estratégica de la producción
- Optimización de redes de distribución
- Programación y mezcla de crudo
- Modelado y optimización de procesos
- Planificación de la utilización de plantas de generación eléctrica
- Gestión estratégica de bosques
- Optimización de aprovisionamientos
- Gestión de riesgo financiero
- Optimización en el diseño de productos
- Planificación de la cartera de I+D y asignación de recursos



CONCLUSIÓN

El análisis detallado de estos cuatro modelos algoritmos evidencia el principal desafío característico a la optimización combinatoria: el crecimiento exponencial del espacio de soluciones conforme aumenta el tamaño de los datos de entrada. Ante la ineficiencia y la imposibilidad computacional de utilizar enfoques de fuerza bruta, resulta indispensable el desarrollo y la aplicación de técnicas avanzadas. Estrategias como el método de Branch and Bound, la recursividad con restricción de espacios de búsqueda, los algoritmos constructivos (como Borůvka, Prim y Kruskal) y los enfoques heurísticos o de inteligencia artificial, proporcionan vías eficientes para evadir la explosion combinatoria. A pesar de que problemas como el JSSP y el TSP pertenecen a clases de alta complejidad computacional, su dominio teórico y representación gráfica siguen siendo herramientas vitales en la ingeniería moderna, permitiendo a las organizaciones reducir redundancias, automatizar procesos logísticos y maximizar el rendimiento de sus recursos en aplicaciones industriales.

REFERENCIAS

[1] Leite, B. S. C. F. (2023, 7 marzo). *The Job-Shop scheduling problem: Mixed-Integer programming models*. Towards Data Science. https://towardsdatascience.com/the-job-shop-scheduling-problem-mixed-integer-programming-models-4bbee83d16ab/

[2] *Learning to Solve Job Shop Scheduling under Uncertainty*. (s. f.). https://arxiv.org/html/2404.01308v1

[3] *Job shop scheduling - Cornell University Computational Optimization Open Textbook - Optimization Wiki. (s. f.).* https://optimization.cbe.cornell.edu/index.php?title=Job_shop_scheduling

[4] Brown, P. (2017, September 5). Solving the n-queens problem. *Medium*. https://medium.com/@paabrown/solving-the-n-queens-problem-6891e3d21b0

[5] Hernandez, P. P. G. (s. f.). *El problema de las reinas*. https://pier.guillen.com.mx/algorithms/09-busqueda/09.7-problema_reinas.htm

[6] *Árbol de expansión mínimo _ AcademiaLab*. (s. f.). https://academia-lab.com/enciclopedia/arbol-de-expansion-minimo/

[7] Benites, L. (s. f.). *Algoritmo de Boruvka (Algoritmo de Sollin)*. https://statologos.com/algoritmo-de-boruvkas/

[8] Benites, L. (s. f.-b). *Árbol de expansión mínimo: definición, ejemplos, algoritmo de Prim*. https://statologos.com/arbol-de-expansion-minimo/

[9] Del Estado de Hidalgo, U. A. (s. f.). *Problema del agente viajero*. https://www.uaeh.edu.mx/scige/boletin/tlahuelilpan/n3/e5.html